

Практична робота № 2 Асимптотична складність алгоритмів. Інші нотації

13. Задано функції $f(n) = n^3 - n^2 + 2$ і $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

$$\frac{f(n)}{g(n)} = \frac{n^3 - n^2 + 2}{n^4} = \frac{1}{n} - \frac{1}{n^2} + \frac{2}{n^4}$$

Тепер візьмемо межу при $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \left(\frac{1}{n} - \frac{1}{n^2} + \frac{2}{n^4} \right) = 0 - 0 + 0 = 0$$

Оскільки $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, то за відомою ознакою асимптотики маємо

$f(n) = o(g(n))$. А оскільки $o(g(n)) \subset O(g(n))$, випливає $n^3 - n^2 + 2 \in O(n^4)$

3. Задано функції $f(n) = 3n^3 - 8n^2 + 15$ та $g(n) = n^3$. Покажіть, що $f(n) = \Omega(g(n))$.

Потрібно знайти сталі $c > 0$ і n_0 такі, що

$$f(n) \geq c \cdot g(n) = cn^3 \text{ для всіх } n \geq n_0.$$

Візьмемо $c=1$. Перевіримо, для яких n виконується $3n^3 - 8n^2 + 15 \geq n^3$.

Це еквівалентно $2n^3 - 8n^2 + 15 \geq 0$.

Для $n \geq 4$ маємо $2n^3 \geq 2 \cdot 4n^2 = 8n^2$. Тому $2n^3 - 8n^2 + 15 \geq 8n^2 - 8n^2 + 15 = 15 > 0$.

Отже для всіх $n \geq 4$ дійсно $2n^3 - 8n^2 + 15 \geq 0$, тобто $f(n) \geq n^3$.

Таким чином можна вибрати $c=1$ і $n_0=4$, і отримаємо $f(n) \geq cn^3 \quad \forall n \geq n_0$.

Отже $f(n) \in \Omega(n^3)$.

Контрольні питання

1. Що таке асимптотична складність алгоритму?

Асимптотична складність алгоритму — це спосіб оцінювання його продуктивності при збільшенні розміру вхідних даних до нескінченності, який показує, як швидко алгоритм працює, коли задача стає великою.

2. Які інші нотації, крім O -нотації, використовуються для вираження асимптотичної складності?

Окрім O-нотації, для вираження асимптотичної складності також використовуються нотація Ω (Омега) та нотація Θ (Тета). Ω -нотація визначає нижню межу складності, тоді як Θ -нотація вказує на точну асимптотичну межу, описуючи як верхню, так і нижню межі водночас.

3. Як визначити асимптотичну складність алгоритму за допомогою символів Θ і Ω ?

Символ Ω ("нижня межа")

Використовується, щоб показати мінімальну швидкість зростання функції.

$f(n) = \Omega(g(n))$, якщо існують константи $c > 0$ і n_0 , такі що

$f(n) \geq c \cdot g(n)$ для всіх $n \geq n_0$

Символ Θ ("точна оцінка")

Використовується для асимптотично точної оцінки функції.

$f(n) = \Theta(g(n))$, якщо існують додатні константи $c_1, c_2 > 0$ і n_0 , такі що

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ для всіх $n \geq n_0$.

4. Яка різниця між O-нотацією, Θ -нотацією і Ω -нотацією?

O-нотація

Верхня межа зростання функції. Показує, наскільки швидко може рости час/пам'ять алгоритму в гіршому випадку.

Ω -нотація

Нижня межа зростання функції. Показує, наскільки повільно принаймні може виконуватись алгоритм.

Θ -нотація

Точна оцінка зростання. Включає і верхню, і нижню межу.

5. Які основні властивості інших нотацій, таких як o (маленька o), ω (маленька омега) та O (маленька o з верхнім індексом)?

о-нотація (маленька о)

Використовується для опису строгої верхньої межі. Якщо $f(n)=o(g(n))$, це означає, що $f(n)$ зростає повільніше, ніж $g(n)$, і ніколи не досягає такого ж порядку.

ω-нотація (маленька омега)

Використовується для опису строгої нижньої межі. Якщо $f(n)=\omega(g(n))$, це означає, що $f(n)$ зростає швидше, ніж $g(n)$, і ніколи не того ж порядку.

Існує кілька різних варіантів використання цієї нотації у різних областях:

У теорії складності алгоритмів часто пишуть $2^{o(n)}$, маючи на увазі, що зростання менше за експоненту 2^{cn} для будь-якого $c>0$. Це означає субекспоненційну складність.